

Số học và đại số tính toán (Number Theory and Algebra)

PHỤ THUỘC A/05 cho số học modulo cơ bản; C/14 cho chia để trị — FFT là ứng dụng sâu nhất của nó; C/20 cho kiểm tra nguyên tố xác suất.

TÓM TẮT

Miền này có một đặc điểm riêng: cạm bẫy chủ yếu không nằm ở ý tưởng mà ở *số học hữu hạn* — tràn số, phép chia không tồn tại, và sự khác biệt giữa số nguyên toán học với số nguyên 64 bit. Chương đi từ thuật toán Euclid và nghịch đảo modulo, qua sàng và phân tích thừa số, tới định lý phần dư Trung Hoa, lũy thừa ma trận, và biến đổi Fourier nhanh. Xuyên suốt là một cách nhìn thống nhất: rất nhiều kỹ thuật ở đây là “đổi biểu diễn để phép toán khó trở nên dễ”, đúng mẫu tư duy đã gặp ở tổng tiền tố (B/06). Nếu chỉ nhớ một điều: mọi lỗi số học trong thi đấu quy về ba nguyên nhân — tràn khi nhân, chia mà không có nghịch đảo, và kết quả âm sau khi lấy dư.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ước chung lớn nhất** — số lớn nhất chia hết cả hai; tính bằng thuật toán Euclid trong $O(\log \min(a, b))$.
- **Euclid mở rộng** — ngoài $\gcd(a, b)$ còn cho hai số x, y với $ax + by = \gcd(a, b)$; đây là cách tính nghịch đảo modulo.
- **nghịch đảo modulo** — số a^{-1} với $a \cdot a^{-1} \equiv 1 \pmod{m}$; tồn tại khi và chỉ khi $\gcd(a, m) = 1$. Thay cho phép chia.
- **sàng Eratosthenes** — đánh dấu bội của từng số nguyên tố để tìm mọi số nguyên tố tới n trong $O(n \log \log n)$.
- **định lý phần dư Trung Hoa** — biết phần dư của x theo nhiều modulo đôi một nguyên tố cùng nhau thì xác định được x theo tích của chúng.
- **biến đổi Fourier nhanh** — thuật toán $O(n \log n)$ tính tích chập hai dãy; ý tưởng là đổi sang biểu diễn mà tích chập trở thành phép nhân từng điểm.

CÂU HỎI MỞ ĐẦU

- Q1. Thuật toán Euclid ra đời hơn hai nghìn năm trước và vẫn được dùng nguyên vẹn. Điều gì làm nó bền tới vậy?
- Q2. Vì sao trong số học modulo, phép chia lại phải thay bằng phép nhân với nghịch đảo, và vì sao nghịch đảo không phải lúc nào cũng tồn tại?
- Q3. Nhân hai đa thức bậc n tốn $O(n^2)$ theo cách thông thường. FFT xuống $O(n \log n)$ bằng cách nào, và ý tưởng đó có họ hàng gì với tổng tiền tố?
- Q4. Vì sao các bài đếm hay yêu cầu modulo 998 244 353 thay vì $10^9 + 7$?
- Q5. Bài toán phân tích một số thành thừa số nguyên tố khó tới mức nào, và vì sao độ khó đó lại là nền của mã hoá hiện đại?

Miền số học có một đặc điểm khiến nó khác các chương trước: phần lớn lỗi ở đây không phải lỗi thuật toán mà là lỗi *số học hữu hạn*. Bạn viết đúng công thức toán học, nhưng máy tính làm

việc với số nguyên 64 bit và với phép chia lấy dư có dấu, và kết quả sai ở những chỗ mà toán học không hề cảnh báo.

Vì vậy chương này sẽ nhấn mạnh ba thứ song song: ý tưởng toán học, cách nó trở thành thuật toán, và *chỗ nó hỏng trên máy thật*. Cách sắp xếp đó phản ánh đúng thứ tự mà bạn sẽ gặp vấn đề trong thực tế.

1.1 Euclid: thuật toán cổ nhất còn dùng

Thuật toán Euclid tính $\gcd(a, b)$ bằng cách thay (a, b) bởi $(b, a \bmod b)$ cho tới khi số thứ hai bằng 0. Tính đúng dựa trên một bất biến rất gọn: $\gcd(a, b) = \gcd(b, a \bmod b)$. Tính đúng dựa trên một đại lượng giảm: số thứ hai giảm nghiêm ngặt. Đây là ví dụ mẫu mực cho bộ đôi công cụ ở chương A/04, và cũng là lý do thuật toán này thường được dùng làm ví dụ đầu tiên khi dạy chứng minh.

Chi phí là $O(\log \min(a, b))$, và trường hợp xấu nhất xảy ra đúng với các số Fibonacci liên tiếp — một liên hệ đẹp giữa lý thuyết số và tổ hợp.

Euclid mở rộng cho thêm hai hệ số x, y thoả $ax + by = \gcd(a, b)$. Ứng dụng quan trọng nhất là tính nghịch đảo modulo: nếu $\gcd(a, m) = 1$ thì $ax + my = 1$, tức $ax \equiv 1 \pmod{m}$, nên x chính là a^{-1} . Đây là câu trả lời cho câu hỏi mở đầu thứ hai: trong số học modulo không có phép chia; “chia cho a ” được định nghĩa là “nhân với nghịch đảo của a ”, và nghịch đảo chỉ tồn tại khi a và m nguyên tố cùng nhau.

Hệ quả thực hành rất cụ thể: khi m là số nguyên tố thì mọi số khác 0 đều có nghịch đảo, nên mọi công thức có phân số đều tính được. Đó chính là lý do đề bài luôn chọn modulo nguyên tố.

1.2 Sàng, phân tích thừa số, và ranh giới của cái dễ

Sàng Eratosthenes tìm mọi số nguyên tố tới n bằng cách đánh dấu bội của từng số nguyên tố. Chi phí là $O(n \log \log n)$ — và con số $\log \log$ này đến từ tổng $\sum_p n/p$ trên các số nguyên tố, một kết quả của lý thuyết số mà chương A/05 đã nhắc.

Hai biến thể đáng biết: sàng lưu *ước nguyên tố nhỏ nhất* của mỗi số, cho phép phân tích thừa số bất kỳ số nào $\leq n$ trong $O(\log n)$; và *sàng tuyến tính*, đảm bảo mỗi hợp số bị đánh dấu đúng một lần, cho $O(n)$ nhưng hằng số không nhỏ hơn nhiều nên hiếm khi cần.

Phân tích một số lớn thì hoàn toàn khác. Thử chia tới $\sqrt{n}$ là $O(\sqrt{n})$ — chấp nhận được tới 10^{12} nhưng không hơn. Thuật toán Pollard rho, một thuật toán ngẫu nhiên đẹp, cho kỳ vọng $O(n^{1/4})$ và xử lý được số 64 bit. Nhưng với số hàng trăm chữ số thì không thuật toán nào đã biết chạy nổi trong thời gian hợp lý.

Ranh giới đó chính là điều đáng nhớ nhất mục này, và nó là câu trả lời cho câu hỏi mở đầu thứ năm. Kiểm tra một số có nguyên tố hay không thì *dễ* (Miller–Rabin, C/20); phân tích nó ra thừa số thì *khó*. Sự bất đối xứng giữa hai bài toán tưởng như anh em ấy là nền của mã hoá khoá công khai: ai cũng nhân được hai số nguyên tố lớn với nhau, nhưng không ai tách ngược lại được.

CẠM BẤY

Ba lỗi số học chiếm gần hết các lần sai trong bài modulo. *Trên khi nhân*: với $m \approx 10^9$, tích hai phần dư đã cỡ 10^{18} , sát giới hạn số nguyên 64 bit có dấu — và nếu $m \approx 10^{18}$ thì bắt buộc phải dùng kỹ thuật nhân đặc biệt. *Kết quả âm*: trong nhiều ngôn ngữ, phép lấy dư của số âm trả về số âm, nên sau phép trừ phải cộng thêm m rồi lấy dư lần nữa. *Chia mà không*

kiểm nghịch đảo: nếu modulo không nguyên tố và số chia không nguyên tố cùng nhau với nó thì nghịch đảo không tồn tại, và công thức của bạn vô nghĩa chứ không chỉ sai số.

1.3 “Kiểm tra thì dễ” – dễ tới mức nào, và từ khi nào

Câu “kiểm tra một số có nguyên tố hay không thì dễ” ở mục trên xứng đáng được nói chính xác hơn, vì đằng sau nó là một trong những kết quả đẹp nhất của lý thuyết tính toán.

Miller–Rabin là thuật toán *ngẫu nhiên*: nó chỉ sai với xác suất nhỏ tùy ý, và trong thực tế thì thế là quá đủ. Nhưng câu hỏi lý thuyết vẫn để ngỏ suốt nhiều thập kỷ: có thuật toán *tất định*, thời gian đa thức, không cần giả thuyết chưa chứng minh nào, để kiểm tra tính nguyên tố hay không? Bản tất định của Miller chạy đa thức nhưng phải dựa vào một giả thuyết lớn của lý thuyết số, nên nó không tính là câu trả lời.

Câu trả lời tới năm 2002, từ Agrawal cùng hai sinh viên của ông là Kayal và Saxena, và được đăng trên *Annals of Mathematics* hai năm sau[1] với một nhan đề gọn tới mức nổi tiếng: “PRIMES is in P”. Thuật toán không dài, không dùng công cụ nặng, và chứng minh vừa trong mười trang. Ý nghĩa của nó với chương này là làm sắc lại đúng sự bất đối xứng đã nói ở mục 2: *kiểm tra* tính nguyên tố nằm trong P một cách vô điều kiện, trong khi *phân tích thừa số* vẫn chưa ai biết cách làm trong thời gian đa thức. Toàn bộ mã hoá khoá công khai sống trên khe hở giữa hai câu ấy.

Và như mọi lần trong quyển sách này, phải kèm câu thứ hai: không ai dùng AKS trong thực tế. Số mũ của nó lớn tới mức Miller–Rabin lặp vài chục vòng nhanh hơn nhiều bậc, với xác suất sai nhỏ hơn xác suất máy tính lỗi phần cứng. Đây là cùng bài học của C/14: kết quả trả lời câu hỏi “có thể hay không”, chứ không phải câu hỏi “nên cài gì”. Người học nên tách bạch hai câu hỏi đó ngay từ đầu, vì trộn chúng là nguồn của cả sự hoài nghi vô lý với lý thuyết lẫn sự sùng bái vô lý với các cặn mới.

1.4 Định lý phần dư Trung Hoa: ghép các mảnh thông tin

Nếu biết $x \equiv a_1 \pmod{m_1}$ và $x \equiv a_2 \pmod{m_2}$ với m_1, m_2 nguyên tố cùng nhau, thì tồn tại duy nhất một x modulo $m_1 m_2$. Ý nghĩa thực dụng: bạn có thể *tách* một phép tính lớn thành nhiều phép tính nhỏ theo các modulo nhỏ, làm độc lập, rồi ghép lại.

Ba ứng dụng hay gặp. Thứ nhất, tránh tràn: thay vì làm việc với modulo 10^{18} , làm hai lần với hai modulo cỡ 10^9 rồi ghép. Thứ hai, tăng độ tin cậy của băm chuỗi (B/08): hai modulo cho xác suất va chạm bằng tích hai xác suất. Thứ ba, trong biến đổi Fourier trên số nguyên khi modulo của bài không có tính chất cần thiết: làm với vài modulo thân thiện rồi ghép.

Nhìn ở mức trừu tượng hơn, đây lại là mẫu quen thuộc: *đổi biểu diễn để phép toán trở nên dễ*. Ta chuyển một số lớn thành bộ phần dư của nó, làm việc trong biểu diễn đó, rồi chuyển ngược. Cùng ý tưởng với tổng tiền tố và mảng hiệu ở B/06, và với FFT ở mục sau.

1.5 Luỹ thừa nhanh và luỹ thừa ma trận

Tính $a^n \bmod m$ bằng cách nhân n lần là $O(n)$; bằng bình phương liên tiếp là $O(\log n)$. Ý tưởng: $a^n = (a^{n/2})^2$ nếu n chẵn, và $a \cdot a^{n-1}$ nếu lẻ — chia để trị ở dạng đơn giản nhất.

Cùng kỹ thuật áp cho ma trận, và đó là công cụ giải các bài “tính trạng thái sau 10^{18} bước”. Điều kiện, như đã nói ở A/05, là quan hệ truy hồi phải *tuyến tính*. Khi đó $v_n = A^n v_0$ và A^n tính được trong $O(d^3 \log n)$ với d là kích thước ma trận.

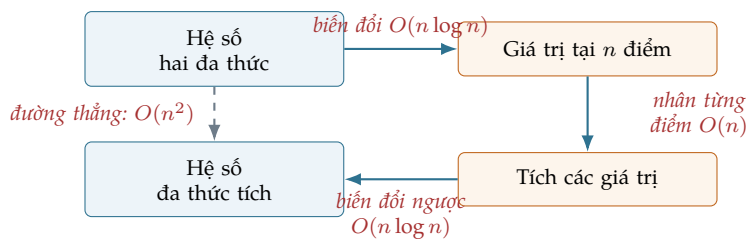
Ba mẹo mở rộng đáng biết. Nếu truy hồi có thêm hằng số cộng, hãy thêm một chiều mang giá trị 1 vào vector trạng thái. Nếu cần *tổng* của dãy, thêm một chiều tích lũy. Và nếu bài hỏi số đường đi độ dài k trong đồ thị, lũy thừa ma trận kể đếm đúng thứ đó — một liên hệ đẹp giữa đại số và đồ thị (C/17).

1.6 Biến đổi Fourier nhanh: đổi miền để nhân trở thành dễ

Bài toán: cho hai đa thức bậc n , tính tích. Cách trường học tốn $O(n^2)$. Karatsuba (C/14) cho $O(n^{1,585})$. FFT cho $O(n \log n)$.

Ý tưởng gồm ba bước, và bước giữa mới là chỗ đáng nhớ. Một đa thức bậc n được xác định hoàn toàn bởi giá trị của nó tại $n + 1$ điểm. Trong *biểu diễn theo giá trị*, phép nhân hai đa thức trở thành phép nhân từng điểm — chỉ $O(n)$. Vậy thuật toán là: đổi cả hai đa thức sang biểu diễn giá trị, nhân từng điểm, rồi đổi ngược về biểu diễn hệ số. Phần khó là làm hai phép đổi đó nhanh, và FFT làm được nhờ chọn các điểm rất đặc biệt — các căn bậc n của đơn vị — khiến phép đổi có cấu trúc chia để trị.

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và nó cho thấy rõ mẫu tư duy chung của cả chương: *phép toán khó ở biểu diễn này lại dễ ở biểu diễn kia; hãy đi đường vòng*. Bạn đã gặp mẫu này ba lần — tổng tiền tố và mảng hiệu, định lý phần dư Trung Hoa, và bây giờ là FFT.



Hình 1.1. Mẫu “đi đường vòng” của cả chương. Phép nhân tốn $O(n^2)$ ở biểu diễn hệ số nhưng chỉ $O(n)$ ở biểu diễn giá trị, nên ta đổi biểu diễn, làm việc, rồi đổi ngược. Cùng mẫu xuất hiện ở tổng tiền tố và mảng hiệu (B/06) và ở định lý phần dư Trung Hoa.

Biến đổi trên số nguyên (NTT) thay các căn phức bằng các căn của đơn vị trong số học modulo. Điều kiện là modulo phải có căn nguyên thủy với bậc là lũy thừa đủ lớn của 2 — và đó chính là lý do 998 244 353 được ưa dùng, vì nó bằng $119 \cdot 2^{23} + 1$. Đây là câu trả lời cho câu hỏi mở đầu thứ tư: modulo đó không phải chọn tùy tiện mà được chọn vì tính chất đại số của nó.

1.7 Đại số trên $\mathbb{F}_2$: khi cộng là XOR

Khi làm việc với các bit, phép cộng modulo 2 chính là XOR, và cả bộ máy đại số tuyến tính vẫn dùng được — chỉ khác là mọi phép toán chạy trên bit nên rất nhanh.

Ứng dụng chính là *cơ sở tuyến tính* của một tập số: duy trì một tập tối thiểu các số sao cho mọi XOR của tập gốc đều biểu diễn được. Với cơ sở đó, ta trả lời được “số này có là XOR của một tập con không”, “XOR lớn nhất có thể là bao nhiêu”, và “có bao nhiêu giá trị XOR phân biệt” — tất cả trong $O(\log C)$ mỗi thao tác.

Cùng khung áp cho giải hệ phương trình tuyến tính trên $\mathbb{F}_2$, thứ xuất hiện trong các bài về đèn và công tắc, và trong lý thuyết mã sửa lỗi.

1.8 Gốc rễ: từ Euclid tới RSA

Chương này có gốc rễ cổ nhất trong cả quyển sách. Thuật toán Euclid xuất hiện trong bộ *Cơ sở* khoảng năm 300 trước Công nguyên, và nó thường được gọi là thuật toán lâu đời nhất còn được dùng nguyên vẹn tới ngày nay — không phải một phiên bản cải tiến, mà đúng thủ tục đó. Sàng Eratosthenes cũng thuộc thời kỳ đó. Điều đáng suy nghĩ: cả hai đều là những thuật toán mà tính đúng dựa trên một bất biến rất gọn, và có lẽ chính sự gọn ấy là lý do chúng sống lâu.

Định lý phần dư Trung Hoa có tên như vậy vì dạng sớm nhất của nó xuất hiện trong một sách toán Trung Quốc khoảng thế kỷ 3–5, dưới hình thức một bài đố: tìm số mà chia 3 dư 2, chia 5 dư 3, chia 7 dư 2. Việc một bài đố dân gian trở thành công cụ tính toán hiện đại là một mẫu lặp lại trong lịch sử toán học.

Bước ngoặt lớn nhất của miền này thì rất gần đây và có tính bước ngoặt thật sự. Suốt hàng thế kỷ, lý thuyết số được coi là ngành toán “thuần túy nhất”, không có ứng dụng — một nhà toán học nổi tiếng đầu thế kỷ 20 còn tự hào về điều đó. Rồi năm 1976–78, ý tưởng mã hoá khoá công khai ra đời, và hệ RSA dựa trực tiếp lên sự bất đối xứng ở mục 2: nhân hai số nguyên tố lớn thì dễ, tách ngược thì khó. Chỉ trong vài năm, những kết quả về số nguyên tố và số học modulo từ chỗ là trò chơi trí tuệ trở thành hạ tầng của thương mại điện tử và của mọi kết nối an toàn trên Internet.

Bài học từ câu chuyện đó vượt ra ngoài số học: *một bài toán được chứng minh là khó có thể trở thành tài nguyên quý*. Trong chương D/23 ta sẽ gặp lại ý này ở dạng tổng quát — độ khó không chỉ là chướng ngại, đôi khi nó là thứ ta cần.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
~300 TCN — Euclid	Tìm ước chung lớn nhất của hai đoạn thẳng (đo lường hình học).	Thuật toán cổ nhất còn dùng nguyên vẹn; tính đúng dựa trên một bất biến, tính dùng dựa trên một đại lượng giảm.
~200 TCN — Eratosthenes	Liệt kê số nguyên tố.	Sàng; vẫn là cách nhanh nhất để tìm mọi số nguyên tố trong một khoảng.
Thế kỷ 3–5 — toán học Trung Quốc	Bài đố: tìm số biết các phần dư.	Định lý phần dư Trung Hoa; nay dùng để tách phép tính lớn thành nhiều phép tính nhỏ.
1640–1801 — Fermat, Euler, Gauss	Hệ thống hoá tính chất của số nguyên tố và đồng dư.	Định lý Fermat nhỏ (nền của cả nghịch đảo modulo lẫn kiểm tra nguyên tố) và ngôn ngữ đồng dư hiện đại.
1965 — Cooley & Tukey	Xử lý tín hiệu địa chấn quy mô lớn.	FFT; ý tưởng “đổi miền để tích chập thành phép nhân từng điểm” trở thành công cụ phổ quát.
1976–78 — mã hoá khoá công khai, RSA	Trao đổi khoá an toàn qua kênh công khai.	Lý thuyết số từ ngành “thuần túy nhất” trở thành hạ tầng của Internet; sự khó của bài phân tích thừa số trở thành tài nguyên.
1975–76 — kiểm tra nguyên tố xác suất	Cần kiểm nguyên tố số hàng trăm chữ số để sinh khoá.	Miller–Rabin; ngẫu nhiên hoá trở thành công cụ sản xuất, không phải giải pháp tạm ($C/20$).

1.9 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Mật mã	RSA, trao đổi khoá, chữ ký số	Dùng đúng các công cụ của chương: lũy thừa modulo, nghịch đảo, kiểm tra nguyên tố. Khác biệt quyết định: ở đó ta dựa vào việc bài toán ngược <i>không</i> giải được nhanh — độ khó là tính năng, không phải lỗi.
Mã sửa lỗi	Reed–Solomon trong mã QR, đĩa quang, truyền tin vệ tinh	Đại số đa thức trên trường hữu hạn — cùng bộ máy với FFT/NTT. Ý tưởng chung: một đa thức bậc k được xác định bởi $k + 1$ điểm, nên gửi dư điểm thì mất vài điểm vẫn khôi phục được.
Xử lý tín hiệu	Tích chập, lọc, phổ tần số	FFT là công cụ nền. Trong thuật toán ta dùng nó để nhân đa thức; trong xử lý tín hiệu, cùng phép biến đổi cho ý nghĩa vật lý là phân tích tần số.
Đồ hoạ và học sâu	Tích chập trong mạng nơ-ron, lọc ảnh	Phép tích chập giống hệt phép nhân đa thức. Trong thực tế, thư viện chọn giữa tích chập trực tiếp và FFT tùy kích thước hạt nhân — đúng bài toán ngưỡng hoà vốn ở $C/14$.
Truyền thông và lưu trữ	Tổng kiểm tra CRC	Số học đa thức trên $\mathbb{F}_2$ — cùng đại số với mục 6. Đây là nơi phép XOR làm việc như phép cộng và toàn bộ lý thuyết vẫn đúng.
Hệ thống nhúng, cảm biến	Số học dấu phẩy tĩnh, tránh chia biến	Cùng mối quan tâm với mục 2: phép chia đắt và có thể mất chính xác, nên người ta thay bằng nhân với nghịch đảo đã tính trước — ý tưởng giống hệt nghịch đảo modulo, chỉ ở miền khác.
Tính toán khoa học	Số học chính xác tùy ý, hệ đại số máy tính	Khi kết quả phải chính xác tuyệt đối — ví dụ kiểm chứng một chứng minh — người ta dùng số nguyên lớn, và mọi thuật toán ở đây phải tính lại chi phí theo số <i>bit</i> thay vì coi mỗi phép toán là $O(1)$ ($A/03$).

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì nó dựa trên một bất biến rất gọn — $\gcd(a, b) = \gcd(b, a \bmod b)$ — và một đại lượng giảm hiển nhiên, nên nó vừa đúng vừa dừng mà không cần giả định gì về dữ liệu. Nó cũng không có tham số nào phải tinh chỉnh, không có trường hợp xấu phụ thuộc dữ liệu, và chi phí $O(\log \min(a, b))$ là gần như tối ưu. Những thuật toán sống lâu thường có chung đặc điểm đó: ít giả định, ít tham số, chứng minh ngắn.
- A2. Vì trong tập các phần dư modulo m , phép chia không được định nghĩa như trên số hữu tỉ — ta chỉ có cộng, trừ, nhân. “Chia cho a ” được định nghĩa lại là “nhân với số a^{-1} sao cho $a \cdot a^{-1} \equiv 1$ ”. Số đó tồn tại khi và chỉ khi $\gcd(a, m) = 1$; nếu a và m có ước chung thì mọi bội của a đều nằm trong một tập con thực sự của các phần dư và không bao giờ chạm tới 1. Đây là lý do modulo nguyên tố được ưa chuộng: khi đó mọi số khác 0 đều nghịch đảo được.
- A3. Bằng cách đối biểu diễn. Một đa thức bậc n được xác định bởi giá trị tại $n + 1$ điểm, và trong biểu diễn theo giá trị thì nhân hai đa thức chỉ là nhân từng điểm, tốn $O(n)$. Toàn bộ việc còn lại là làm hai phép đối biểu diễn cho nhanh, và FFT làm được nhờ chọn các điểm là căn của đơn vị, khiến phép đối có cấu trúc chia đôi đệ quy. Họ hàng với tổng tiên tố ở chỗ cả hai đều là “đi đường vòng qua một biểu diễn khác, nơi phép toán cần làm trở thành rẻ”.
- A4. Vì $998\,244\,353 = 119 \cdot 2^{23} + 1$, nên trong số học modulo đó tồn tại căn của đơn vị bậc 2^{23} — điều kiện để chạy biến đổi Fourier trên số nguyên với độ dài là lũy thừa của 2 tới 2^{23} . Modulo $10^9 + 7$ không có tính chất đó, nên các bài cần nhân đa thức lớn buộc phải dùng modulo thân thiện với biến đổi. Nói cách khác, đó là một lựa chọn kỹ thuật có lý do đại số cụ thể, không phải quy ước tùy tiện.

- A5.** Rất khó theo nghĩa thực tiễn: với số hàng trăm chữ số, không thuật toán công khai nào phân tích được trong thời gian hợp lý bằng máy tính cổ điển, dù bài toán này đã được nghiên cứu hàng thế kỷ. Đồng thời, *kiểm tra* một số có nguyên tố hay không lại dễ. Sự bất đối xứng đó cho phép xây một hệ mã mà ai cũng mã hoá được (nhân hai số nguyên tố) nhưng chỉ người giữ khoá mới giải được (biết hai thừa số). Cần lưu ý: đây là độ khó *chưa ai vượt qua*, không phải độ khó đã được chứng minh — chương D/23 sẽ nói vì sao sự phân biệt đó quan trọng.

NÓI LẠI THẬT ĐƠN GIẢN

Số học trong lập trình có một luật chơi khác với toán học ở trường: mọi con số đều bị nhốt trong một cái vòng. Nếu vòng có 12 vạch như mặt đồng hồ thì 10 cộng 5 không ra 15 mà ra 3. Cộng, trừ, nhân đều chạy bình thường trong cái vòng đó. Chỉ có phép chia là rắc rối, vì “chia 2” bây giờ có nghĩa là “nhân với cái gì đó để quay về đúng vị trí ban đầu”, và cái gì đó không phải lúc nào cũng tồn tại. Nó tồn tại khi số bạn chia không có ước chung nào với kích thước vòng — đó là lý do người ta luôn chọn kích thước vòng là số nguyên tố.

Thuật toán cổ nhất còn được dùng tới hôm nay cũng nằm trong chương này, và nó đơn giản tới bất ngờ: muốn tìm ước chung lớn nhất của hai số, cứ lấy số lớn chia số bé rồi thay số lớn bằng phần dư, lặp lại tới khi dư bằng 0. Hai nghìn ba trăm năm và không ai cải tiến được đáng kể.

Còn ý tưởng sâu nhất của chương thì là chuyện đi đường vòng. Nhân hai đa thức dài thì lâu, nhưng nếu thay vì giữ chúng dưới dạng danh sách hệ số, ta ghi lại giá trị của chúng tại một loạt điểm, thì nhân chỉ còn là nhân từng cặp giá trị — rất nhanh. Vấn đề chuyển thành: làm sao đổi qua đổi lại giữa hai cách ghi cho nhanh. Câu trả lời là chọn các điểm thật khéo, và đó là toàn bộ nội dung của phép biến đổi Fourier nhanh.

Cuối cùng là một sự thật đáng nhớ: nhân hai số nguyên tố lớn thì máy tính làm trong tích tắc, nhưng tách ngược một tích ra hai thừa số thì cả thế giới chưa ai làm nổi khi số đủ lớn. Toàn bộ việc mua bán an toàn trên mạng dựa vào đúng sự chênh lệch đó.

Một câu: Trong số học máy tính, mọi thứ chạy trong một cái vòng — và phần lớn lỗi đến từ việc quên mất điều đó.

Chỗ không rút gọn được: Vì sao phân tích thừa số khó thì chưa ai chứng minh được; ta chỉ biết là chưa ai làm được.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Thuật toán Euclid	$\gcd(a, b) = \gcd(b, a \bmod b)$; bất biến cho tính đúng, số thứ hai giảm cho tính dừng; $O(\log \min(a, b))$.
••• Nghịch đảo modulo	Thay cho phép chia; tồn tại khi $\gcd(a, m) = 1$; tính bằng Euclid mở rộng hoặc bằng a^{m-2} khi m nguyên tố.
••• Ba lỗi số học kinh điển	Trần khi nhân hai phần dư cỡ 10^9 ; kết quả âm sau phép trừ rồi lấy dư; chia mà nghịch đảo không tồn tại.
••• Sàng Eratosthenes	$O(n \log \log n)$; biến thể lưu ước nguyên tố nhỏ nhất cho phép phân tích mọi số $\leq n$ trong $O(\log n)$.
••• Bất đối xứng kiểm tra / phân tích	Kiểm nguyên tố thì dễ (Miller–Rabin); phân tích thừa số thì khó — nền của mã hoá khoá công khai.
••• Luỹ thừa nhanh	$a^n = (a^{n/2})^2$; $O(\log n)$. Áp cho ma trận cho các bài truy hồi tuyến tính với n tới 10^{18} .
•• Định lý phần dư Trung Hoa	Phần dư theo các modulo nguyên tố cùng nhau xác định duy nhất số theo tích; dùng để tách phép tính lớn thành nhiều phép tính nhỏ.
•• Ý tưởng của FFT	Đa thức bậc n xác định bởi $n + 1$ giá trị; trong biểu diễn giá trị, nhân là nhân từng điểm $O(n)$; chọn căn của đơn vị để đổi biểu diễn nhanh.
•• Vì sao 998 244 353	Bằng $119 \cdot 2^{23} + 1$ nên có căn của đơn vị bậc 2^{23} — điều kiện chạy biến đổi Fourier trên số nguyên.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Cơ sở tuyến tính trên $\mathbb{F}_2$	Cộng là XOR; duy trì tập tối thiểu để trả lời “có biểu diễn được không”, “XOR lớn nhất”, “bao nhiêu giá trị phân biệt”.
●● Mẫu tư duy chung của chương	Đổi biểu diễn để phép toán khó thành dễ: tổng tiền tố, phần dư Trung Hoa, FFT — ba lần cùng một ý.
● Trường hợp xấu của Euclid	Hai số Fibonacci liên tiếp — một liên hệ giữa lý thuyết số và tổ hợp.
● PRIMES is in P	Kiểm tra tính nguyên tố có thuật toán tất định, đa thức, vô điều kiện (2004); phân tích thừa số thì chưa ai biết. Khe hở giữa hai câu đó là nền của mã hoá khoá công khai. Thực tế vẫn dùng Miller–Rabin.

●●● phải thuộc ●● nên nắm ● biết là có

▮ Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tính $\binom{n}{k} \bmod p$ cho n tới 10^6 thì làm thế nào?

| Tiên xử lý mảng giai thừa và mảng nghịch đảo giai thừa một lần trong $O(n)$, rồi mỗi truy vấn là một phép nhân ba số trong $O(1)$. Mẹo tính nghịch đảo giai thừa nhanh: tính nghịch đảo của $n!$ một lần bằng lũy thừa, rồi đi ngược $\text{invfact}[i-1] = \text{invfact}[i] \cdot i$. Khi n lớn hơn p thì công thức này không dùng được và cần định lý Lucas.

2. Modulo trong bài của tôi không nguyên tố. Có cách nào tính phân số không?

| Có ba hướng. Nếu số chia nguyên tố cùng nhau với modulo thì nghịch đảo vẫn tồn tại, tính bằng Euclid mở rộng. Nếu không, hãy thử tách modulo thành các lũy thừa nguyên tố, tính theo từng lũy thừa rồi ghép bằng định lý phần dư Trung Hoa. Hoặc, thường là cách tốt nhất, hãy sắp xếp lại công thức để tránh phép chia — ví dụ tính tổ hợp bằng truy hồi Pascal thay vì bằng giai thừa.

3. Làm sao nhân hai số modulo 10^{18} mà không tràn?

| Ba cách. Dùng kiểu số nguyên 128 bit nếu môi trường hỗ trợ — đơn giản nhất. Dùng phép nhân kiểu “nhân bằng cách nhân đôi” theo bit, $O(\log m)$ nhưng chắc chắn. Hoặc dùng mẹo nhân qua số thực dấu phẩy động để ước lượng thương rồi hiệu chỉnh — nhanh nhưng cần cẩn thận về độ chính xác. Trong thi đấu, kiểu 128 bit là lựa chọn mặc định khi có.

4. Khi nào tôi thực sự cần FFT?

| Khi phải tính tích chập của hai dãy dài hơn vài nghìn phần tử — nhân đa thức, đếm số cặp có tổng bằng mỗi giá trị, nhân số lớn. Dưới ngưỡng đó, cách trực tiếp $O(n^2)$ thường nhanh hơn vì hằng số nhỏ hơn nhiều. Dấu hiệu trên đề: bài yêu cầu, với mỗi giá trị s , đếm số cặp (i, j) có $a_i + b_j = s$ — đó chính xác là tích chập.

5. Kiểm tra một số tới 10^{18} có nguyên tố không thì dùng gì?

| Miller–Rabin với một tập nhân chứng cố định đã được kiểm chứng là đủ cho mọi số 64 bit — đây là một trong số ít trường hợp mà thuật toán ngẫu nhiên được “khử ngẫu nhiên” bằng một danh sách nhân chứng cụ thể. Nếu cần phân tích thừa số chứ không chỉ kiểm tra, hãy kết hợp Miller–Rabin với Pollard rho.

6. Bài yêu cầu tính $a^b \bmod m$ với b là số có 10^5 chữ số. Làm sao?

| Dùng định lý Euler: nếu $\gcd(a, m) = 1$ thì $a^{\varphi(m)} \equiv 1$, nên có thể rút gọn số mũ theo modulo $\varphi(m)$. Khi $\gcd(a, m) \neq 1$ thì phải dùng phiên bản mở rộng của định lý, với điều kiện số mũ đủ lớn. Đây là một trong những chỗ dễ sai nhất của chương vì trường hợp biên rất tinh vi — hãy stress test với vết cặn ở m nhỏ.

7. Vì sao chương này lại xếp vào phần “Giới hạn” cùng với NP-khó?

Vì nó là nơi ta gặp lần đầu một bài toán mà *độ khó là điều đáng mong muốn*. Trong toàn bộ phần A–C, khó là chương ngại. Ở đây, sự khó của bài phân tích thừa số là thứ giữ an toàn cho mọi giao dịch trên mạng. Cách nhìn đó chuẩn bị cho chương D/23, nơi ta học cách *nhận ra* một bài toán là khó và coi đó là thông tin có giá trị chứ không phải thất bại.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Euclid và Euclid mở rộng, giải phương trình $ax + by = c$	CSES “Exponentiation”, bài kinh điển[2]	Chú ý điều kiện có nghiệm và cách sinh mọi nghiệm.
● Luỹ thừa nhanh modulo	CSES “Exponentiation”[2]	Chú ý tràn khi nhân và trường hợp số mũ bằng 0.
● Sàng số nguyên tố và đếm ước	CSES “Counting Divisors”[2]	Sàng lưu ước nguyên tố nhỏ nhất; so với cách thử chia từng số.
●● Tính tổ hợp modulo với truy vấn nhiều	CSES “Binomial Coefficients”[2]	Tiền xử lý giai thừa và nghịch đảo giai thừa; mỗi truy vấn $O(1)$.
●● Fibonacci thứ 10^{18} bằng luỹ thừa ma trận	CSES “Fibonacci Numbers”[2]	Kiểm chứng với giá trị nhỏ; chú ý thứ tự nhân ma trận.
●● Phân tích thừa số tới 10^{18}	Bài kinh điển	Miller–Rabin cộng Pollard rho; kiểm bằng cách nhân lại các thừa số.
●● Giải hệ đồng dư bằng định lý phần dư Trung Hoa	Bài kinh điển	Trường hợp modulo không nguyên tố cùng nhau cần xử lý riêng — đừng bỏ qua.
●●● Nhân hai đa thức bằng NTT	Bài kinh điển	Modulo 998 244 353; so thời gian với cách $O(n^2)$ để tìm ngưỡng hoà vốn.
●●● XOR lớn nhất của một tập con	Bài kinh điển	Cơ sở tuyến tính trên $\mathbb{F}_2$; thuật toán tham lam từ bit cao xuống thấp sau khi có cơ sở.
●●● Đếm số cặp có tổng bằng mỗi giá trị	Bài kinh điển	Nhận ra đây là tích chập; đây là bài mẫu để nhận diện dấu hiệu dùng FFT.
●●● Tính $a^b \bmod m$ với b rất lớn	Bài kinh điển	Định lý Euler và phiên bản mở rộng; stress test với m nhỏ vì trường hợp biên rất tinh vi.

Kết chương và đọc tiếp

Chương này là miền mà lỗi hay nằm ở số học chứ không ở ý tưởng, nên ba lỗi ở khối *Cạm bẫy* đáng thuộc hơn bất kỳ công thức nào. Hai điều nữa đáng mang theo: mẫu “đổi biểu diễn để phép toán trở nên dễ”, xuất hiện ba lần trong chương và nhiều lần trong cả quyển sách; và sự bất đối xứng giữa kiểm tra và tìm kiếm, thứ vừa là nền của mật mã vừa là lời mở đầu cho chương D/23.

Chương tiếp theo là miền chuyên thứ hai, và nó có cùng đặc điểm: cạm bẫy không nằm ở ý tưởng mà ở số học. Chỉ khác là ở đó, thủ phạm không phải phép chia dư mà là số dấu phẩy động — và hậu quả nghiêm trọng hơn, vì một phép so sánh sai có thể làm cả thuật toán hình học rơi vào trạng thái không nhất quán (D/22).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Agrawal, Kayal, Saxena, "PRIMES Is in P"	Annals of Mathematics, 2004	Mười trang, sơ cấp một cách đáng ngạc nhiên. Đọc phần mở đầu và phát biểu thuật toán, kể cả khi bỏ qua chứng minh.
nền tảng	Cooley & Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series"	Mathematics of Computation, 1965	Bài gốc của FFT, nền của mục 5. Ngắn, và cho thấy động cơ ban đầu đến từ xử lý tín hiệu.
tra khi cần	Harvey & van der Hoeven, "Integer Multiplication in Time $O(n \log n)$ "	Annals of Mathematics, 2021	Đọc phần mở đầu để nối mục 5 của chương này với biên giới hiện nay của phép nhân ($\mathbb{C}/14$).
tra khi cần	Graham, Knuth, Patashnik, <i>Concrete Mathematics</i>	Addison-Wesley, tái bản 1994	Tra khi cần các tổng và đồng dư xuất hiện trong phân tích; chương về lý thuyết số sơ cấp viết rất kỹ.

Nguồn

- [1] M. Agrawal, N. Kayal, and N. Saxena, “PRIMES Is in P,” *Annals of Mathematics*, vol. 160, no. 2, pp. 781–793, 2004.
- [2] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).